

LAPORAN
MICROSERVICES ARCHITECTURE

Matkul: Praktikum Sistem Terdistribusi



Disusun Oleh Kelompok 2 :

Shavira Nindya Putriawan	2E TRPL	362458302050
Rahma Titis Pratiwi	2E TRPL	362458302052
Moh Andy Yusril	2E TRPL	362458302111
Naura Ulfatin Nadya	2E TRPL	362458302129

Dosen Pengampu:

Furiansyah Dipraja S.T., M.T

PROGRAM STUDI TEKNOLOGI REKAYASA PERANGKAT LUNAK
JURUSAN BISNIS DAN INFORMATIKA
POLITEKNIK NEGERI BANYUWANGI

2025/2026

DAFTAR ISI

Contents

BAB I	3
PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan Praktikum.....	3
1.3 Ruang Lingkup	3
BAB II	4
DASAR TEORI.....	4
2.1 Microservices Architecture	4
2.2 Perbedaan Monolitik dan Microservices.....	4
2.3 API Gateway	4
2.4 Containerization (Docker)	5
BAB III.....	6
3.1 Arsitektur Sistem	6
3.2 Diagram Arsitektur Microservices.....	6
3.3 Penjelasan Tiap Service.....	6
BAB IV.....	7
4.1 Tools	7

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini menuntut sistem komputer untuk mampu menangani data dalam jumlah besar serta melayani banyak pengguna secara bersamaan. Pada awalnya, banyak sistem dibangun menggunakan arsitektur monolitik, yaitu seluruh fitur dan fungsi aplikasi digabung dalam satu kesatuan.

Namun, seiring bertambahnya kompleksitas sistem, pendekatan monolitik mulai menunjukkan berbagai keterbatasan. Proses pengembangan menjadi lebih lambat, perubahan kecil dapat berdampak pada seluruh sistem, serta sulit untuk melakukan scaling secara efisien.

Sebagai solusi dari permasalahan tersebut, muncul konsep Microservices Architecture. Microservices merupakan pendekatan dalam pengembangan perangkat lunak yang memecah aplikasi menjadi layanan-layanan kecil yang berdiri sendiri dan saling terhubung. Setiap layanan memiliki tanggung jawab spesifik dan dapat dikembangkan, diuji, serta dijalankan secara independen.

Pendekatan ini banyak digunakan dalam sistem modern karena memberikan fleksibilitas tinggi, kemudahan dalam pengembangan, serta mendukung skalabilitas yang lebih baik.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

- Memahami konsep dasar Microservices Architecture
- Mengetahui perbedaan antara monolitik dan microservices
- Mempelajari cara kerja sistem terdistribusi
- Mengimplementasikan layanan microservices sederhana
- Melakukan pengujian terhadap sistem yang dibuat

1.3 Ruang Lingkup

- Pembuatan beberapa service terpisah
- Komunikasi antar service menggunakan API
- Penggunaan Docker untuk containerization
- Implementasi monitoring dan cache

BAB II

DASAR TEORI

2.1 Microservices Architecture

Microservices Architecture adalah pendekatan dalam pengembangan aplikasi di mana sistem dibagi menjadi beberapa layanan kecil (services) yang saling terhubung melalui jaringan. Setiap service memiliki fungsi tertentu dan dapat berjalan secara mandiri.

Menurut konsep yang diperkenalkan oleh para praktisi seperti Martin Fowler, microservices memungkinkan setiap bagian aplikasi dikembangkan secara terpisah tanpa harus bergantung pada keseluruhan sistem.

Ciri utama microservices:

- Setiap service memiliki tanggung jawab spesifik
- Berjalan secara independen
- Komunikasi melalui API
- Dapat dikembangkan dan di-deploy secara terpisah

2.2 Perbedaan Monolitik dan Microservices

Pada arsitektur monolitik, seluruh komponen aplikasi berada dalam satu sistem. Hal ini membuat sistem lebih sederhana di awal, namun sulit dikembangkan ketika skala meningkat.

Sedangkan pada microservices:

- Sistem dibagi menjadi beberapa bagian kecil
- Lebih mudah di-scale
- Perubahan tidak mempengaruhi seluruh sistem
- Lebih fleksibel dalam penggunaan teknologi

2.3 API Gateway

API Gateway merupakan komponen penting dalam microservices yang berfungsi sebagai pintu masuk utama bagi client. Semua request dari pengguna akan melalui gateway sebelum diteruskan ke service yang sesuai.

Fungsi API Gateway:

- Routing request
- Autentikasi
- Monitoring
- Load balancing sederhana

2.4 Containerization (Docker)

Docker adalah teknologi yang digunakan untuk menjalankan aplikasi dalam container. Container memungkinkan aplikasi berjalan dengan environment yang konsisten di berbagai perangkat.

Keuntungan penggunaan Docker:

- Mudah deployment
- Konsisten antar environment
- Mendukung microservices
- Mengurangi beban request langsung
- Meningkatkan keandalan sistem

BAB III

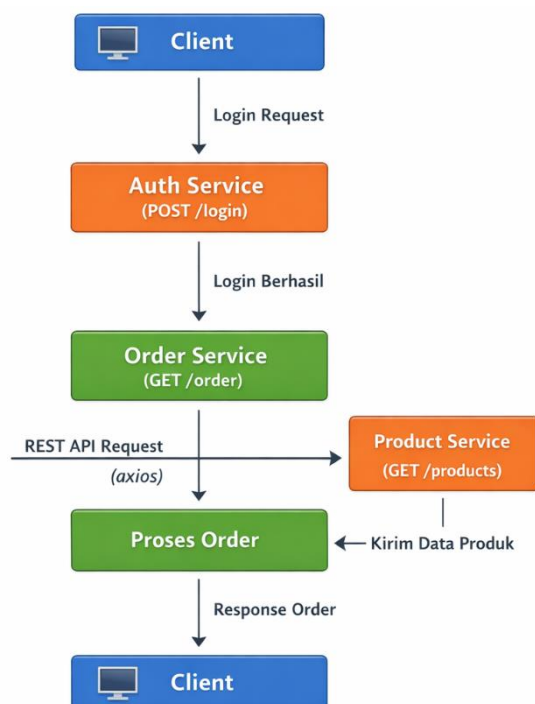
ARSITEKTUR SISTEM

3.1 Arsitektur Sistem

Pada praktikum ini, sistem dibangun menggunakan pendekatan microservices yang terdiri dari beberapa service utama:

- **Auth Service** → menangani login/auth
- **Product Service** → menyediakan data produk
- **Order Service** → memproses order dan mengambil data dari Product Service. Setiap service berjalan secara terpisah dan saling berkomunikasi menggunakan REST API.

3.2 Diagram Arsitektur Microservices



Alur ini menggambarkan komunikasi **synchronous** antar service, di mana setiap service menunggu respons dari service lain sebelum melanjutkan prosesnya. Diagram ini mencerminkan struktur project, yaitu adanya tiga service utama: Auth Service, Product Service, dan Order Service, serta mekanisme interaksi mereka melalui REST API.

Diagram ini menunjukkan alur sistem microservices. Client mengirim request login ke Auth Service (POST /login) dan menerima respons login berhasil. Kemudian client membuat order melalui Order Service (GET /order), yang mengambil data produk dari Product Service (GET /products) menggunakan Axios. Setelah memproses order, Order Service mengirimkan respons kembali ke client.

3.3 Penjelasan Tiap Service

- **User Service:** mengelola data pengguna
- **Product Service:** mengelola data produk
- **Transaction Service:** mengelola transaksi
- **API Gateway:** penghubung client dengan service

BAB IV

IMPLEMENTASI

4.1 Tools

Sistem Microservices menggunakan teknologi berikut:

- **Node.js** – digunakan untuk membangun setiap service secara independen
- **Express.js** – framework untuk membuat REST API pada masing-masing service
- **Docker** – digunakan untuk menjalankan setiap service dalam container
- **Docker Compose** – untuk menjalankan beberapa service secara bersamaan
- **Postman / HTTP Client** – untuk melakukan pengujian endpoint API
- **Axios** – digunakan untuk komunikasi antar service (REST API)

4.2 Deskripsi Sistem

Praktikum ini bertujuan untuk membangun sistem sederhana berbasis **Microservices Architecture** menggunakan REST API. **NOTE: Sistem menggunakan komunikasi, Synchronous (REST API).** *Artinya: Setiap service harus menunggu respon dari service lain sebelum melanjutkan proses*

4.3. Persiapan lingkungan

○ BUAT PROJECT

```
D:\>mkdir Praktikum-Micro-Architecture
D:\>cd Praktikum-Micro-Architecture
D:\Praktikum-Micro-Architecture>code .
D:\Praktikum-Micro-Architecture>mkdir auth-service
D:\Praktikum-Micro-Architecture>mkdir product-service
D:\Praktikum-Micro-Architecture>mkdir order-service
D:\Praktikum-Micro-Architecture>cd auth-service
```

○ Struktur folder

```
Praktikum-Micro-Architecture/
├── auth-service/
│   ├── index.js
│   ├── package.json
│   ├── package-lock.json
│   └── Dockerfile
├── product-service/
│   ├── index.js
│   ├── package.json
│   ├── package-lock.json
│   └── Dockerfile
├── order-service/
│   ├── index.js
│   ├── package.json
│   ├── package-lock.json
│   └── Dockerfile
└── docker-compose.yml
```

- auth-service: Berisi service untuk autentikasi user (login)
- product-service: Berisi data produk yang akan digunakan oleh order-service.
- order-service: Berfungsi untuk memproses order dan mengambil data dari product-service.

- lalu masuk ke setiap folder service dan jalankan

```
D:\Praktikum-Micro-Architecture>cd auth-service

D:\Praktikum-Micro-Architecture\auth-service>npm init -y
Wrote to D:\Praktikum-Micro-Architecture\auth-service\package.json

{
  "name": "auth-service",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

D:\Praktikum-Micro-Architecture\auth-service>npm install express
added 65 packages, and audited 66 packages in 25s
```

Setiap service memiliki dependency yang berbeda karena dalam arsitektur microservices, setiap service bersifat independen dan hanya menginstall library yang dibutuhkan sesuai fungsinya masing-masing. Misal: pada auth-service dan product-service hanya menggunakan npm install express karena Tidak butuh data dari service lain sedangkan Order Service BUTUH data dari service lain (Product Service) maka menggunakan axios.

```
D:\Praktikum-Micro-Architecture\order-service>npm install express axios
added 76 packages, and audited 77 packages in 4s
```

- Membuat Endpoint Tiap Service

folder	code	fungsi
Auth Service	<pre>app.post("/login", (req, res) => { const { username, password } = req.body;</pre>	Untuk autentikasi user
Product Service	<pre>app.get("/products", (req, res) => { res.json([{ id: 1, name: "Produk A", price: 10000 }]); });</pre>	Menyediakan data produk
Order Service	<pre>app.get('/order', async (req, res) => { try { // ♦ Ambil data dari product-service const product = await axios.get('http://product-service:3001/products');</pre>	Menyediakan data produk

- membuat docker file untuk semua folder

```

order-service > Dockerfile
1 FROM node:18
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9
10 CMD ["node", "index.js"]

```

tujuannya: Membungkus aplikasi ke dalam container

- o Membuat docker-compose.yml -> Menjalankan semua service sekaligus

```

docker-compose.yml
1 services:
2   auth-service:
3     build: ./auth-service
4     ports:
5       - "3000:3000"
6
7   product-service:
8     build: ./product-service
9     ports:
10      - "3001:3001"
11
12   order-service:
13     build: ./order-service
14     ports:
15       - "3002:3002"
16     depends_on:
17       - product-service
18       - auth-service
19

```

- o Menjalankan Project

```

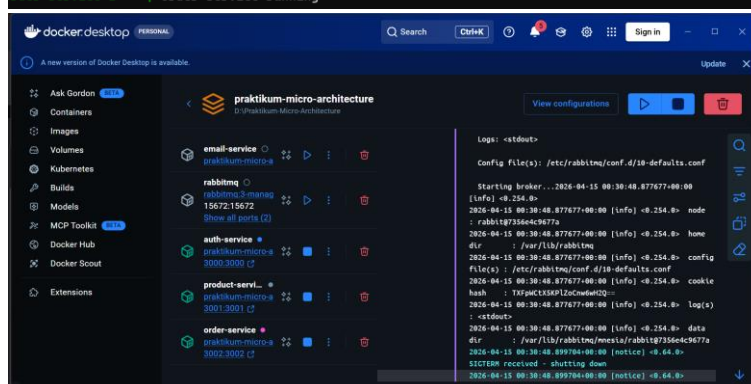
D:\Praktikum-Micro-Architecture>docker compose up --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.68kB done
#1 DONE 0.0s

```

```

✓Container praktikum-micro-architecture-product-service-1 Recreated 2.7s
✓Container praktikum-micro-architecture-auth-service-1 Recreated 2.7s
✓Container praktikum-micro-architecture-order-service-1 Recreated 0.5s
Attaching to auth-service-1, order-service-1, product-service-1
auth-service-1 | Auth Service running
product-service-1 | Product Service running on port 3001
order-service-1 | Order Service running

```



- mengecek apakah semua service berjalan?

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
a142dd1f8a42	praktikum-micro-architecture-order-service	"docker-entrypoint.s..."	24 seconds ago	Up 22 seconds	0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
a278057518ca	praktikum-micro-architecture-auth-service	"docker-entrypoint.s..."	27 seconds ago	Up 23 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
eel16251e509	praktikum-micro-architecture-product-service	"docker-entrypoint.s..."	27 seconds ago	Up 23 seconds	0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp

Jika berhasil, akan terlihat semua container aktif.

BAB V

PENGUJIAN SISTEM (API GATEWAY MICROSERVICE)

5.1 Konsep Pengujian Microservices

Pada arsitektur microservices, pengujian dilakukan untuk memastikan bahwa setiap service dapat berjalan dengan baik serta komunikasi antar service dapat berjalan dengan benar.

Pengujian meliputi:

- Service Testing
- Integration Testing
- Error Handling

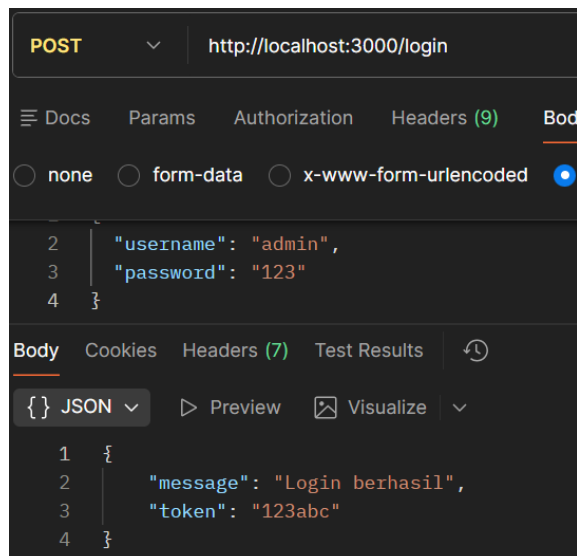
5.2 Arsitektur yang Diuji

Auth-service	http://localhost:3000
Product-service	http://localhost:3001
Order-service	http://localhost:3002

Komunikasi antar service menggunakan REST API.

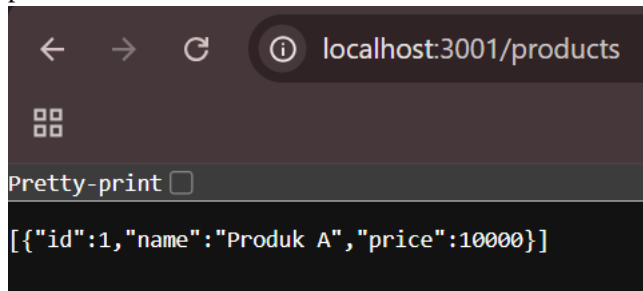
5.3 Service Testing

- auth-service



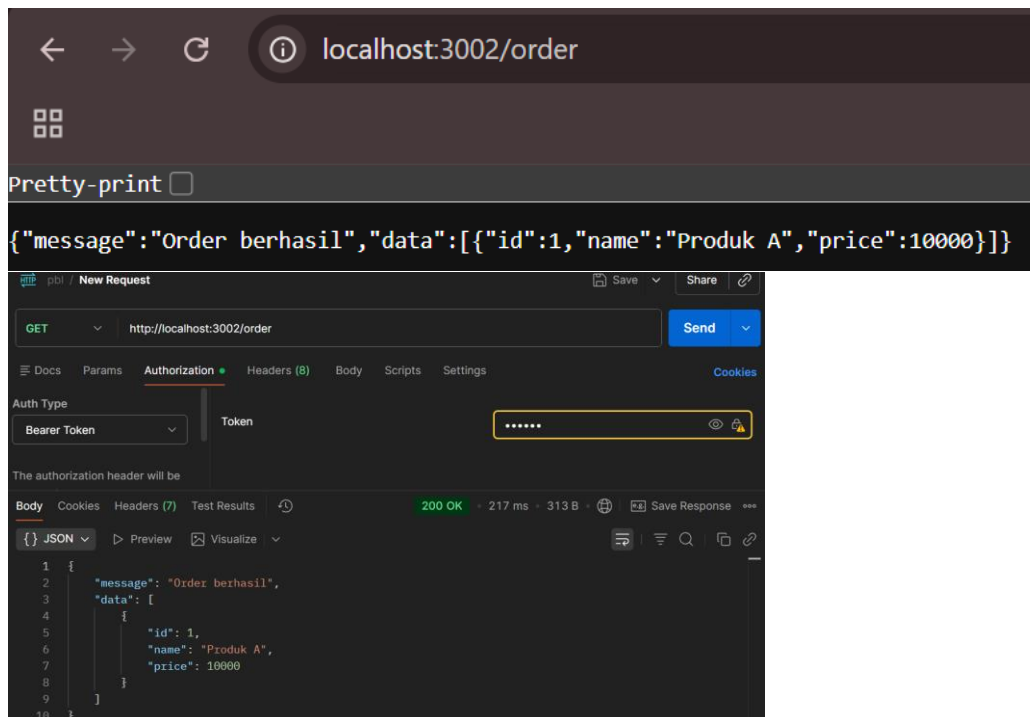
Auth Service berhasil melakukan proses login. Endpoint login menggunakan metode POST sehingga tidak dapat diakses langsung melalui browser, melainkan harus menggunakan tools seperti Postman atau HTTP client untuk mengirim data request.

- product-service



Product Service berhasil menampilkan data produk. Product Service berhasil menampilkan data produk dengan menggunakan metode GET. Data produk dapat diakses dan ditampilkan dengan baik melalui endpoint yang telah disediakan.

- order-service



Order Service berhasil melakukan proses pembuatan order menggunakan metode POST serta menampilkan data order melalui metode GET.

Selain itu, Order Service juga berhasil melakukan komunikasi dengan Product Service menggunakan REST API untuk mengambil data produk.

KESIMPULAN

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem Microservices Architecture berhasil dibangun dengan memisahkan aplikasi menjadi beberapa service, yaitu Auth Service, Product Service, dan Order Service. Masing-masing service memiliki fungsi yang berbeda, di mana Auth Service digunakan untuk proses login pengguna, Product Service berfungsi untuk mengelola dan menyediakan data produk, sedangkan Order Service digunakan untuk membuat serta mengelola data pesanan.

Penerapan konsep microservices ini membuat setiap service dapat berjalan secara mandiri tanpa saling bergantung secara langsung, namun tetap dapat saling terhubung melalui REST API. Hal ini terlihat dari keberhasilan komunikasi antar service, khususnya pada Order Service yang dapat mengambil data dari Product Service menggunakan Axios. Dengan pemisahan seperti ini, sistem menjadi lebih mudah dikembangkan, lebih fleksibel, dan lebih mudah dalam proses pemeliharaan.

Hasil pengujian menggunakan Postman juga menunjukkan bahwa seluruh endpoint pada setiap service dapat berjalan dengan baik, mulai dari proses login pada Auth Service, pengambilan data produk pada Product Service, hingga proses pembuatan dan pengambilan data order pada Order Service. Selain itu, sistem ini juga sudah menggambarkan konsep dasar komunikasi antar service pada arsitektur terdistribusi.

Namun, pada implementasi sederhana ini data order masih disimpan di dalam memory (array), sehingga data tidak bersifat permanen dan akan hilang ketika service dihentikan atau dijalankan ulang.

Hal ini menjadi salah satu kekurangan sistem yang masih bisa dikembangkan lebih lanjut. Secara keseluruhan, implementasi Microservices Architecture ini sudah sesuai dengan konsep dasar sistem terdistribusi, yaitu pemisahan layanan, kemandirian setiap service, serta komunikasi antar service menggunakan API. Sistem ini juga memberikan gambaran bagaimana aplikasi modern dibangun dengan pendekatan microservices yang lebih modular dan scalable dibandingkan arsitektur monolitik.